

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import LSTM, Dense

# Load the dataset
df = pd.read_csv('/content/stock_data (1).csv')
df.head()
```



	Date	Close
0	2024-01-01	148.750108
1	2024-01-02	151.082577
2	2024-01-03	149.136624
3	2024-01-04	151.178820
4	2024-01-05	151.544438




Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
plt.figure(figsize=(6,4))
plt.plot(df['Close'])
plt.title('Stock Closing Price')
plt.xlabel('Date')
plt.ylabel('Price')
plt.show()

# Preprocessing data
scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(df[['Close']])

# Create training and testing data
train_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[:train_size]
test_data = scaled_data[train_size:]

def create_sequences(data, seq_length):
    x, y = [], []
    for i in range(seq_length, len(data)):
        x.append(data[i-seq_length:i, 0])
        y.append(data[i, 0])
    return np.array(x), np.array(y)

seq_length = 60 # Use the past 60 days
x_train, y_train = create_sequences(train_data, seq_length)
x_test, y_test = create_sequences(test_data, seq_length)

# Reshape for LSTM [samples, time steps, features]
x_train = x_train.reshape((x_train.shape[0], x_train.shape[1], 1))
print(f"x_test.shape before reshape: {x_test.shape}")
if len(x_test.shape) == 1 or x_test.shape[0] == 0:
    print("Error: x_test is empty or not enough data points for sequence length.")
    print(f"Length of test_data: {len(test_data)}")
    if len(test_data) < seq_length:
        print("Not enough data in test set. Please use more data or reduce sequence length.")
    if len(x_test.shape) < 2 or x_test.shape[0] == 0:
        print("Error: x_test is empty. Cannot reshape.")
    x_test = np.zeros((1, seq_length, 1)) # Dummy fallback to avoid crashing
else:
    x_test = x_test.reshape((x_test.shape[0], x_test.shape[1], 1))

if x_test.shape[0] == 0:
    raise ValueError(f"x_test is empty. Check your test data size or reduce seq_length (currently {60}).")
```

```

else:
    x_test = x_test.reshape((x_test.shape[0], x_test.shape[1], 1))

# Build LSTM Model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(x_train.shape[1],1)))
model.add(LSTM(units=50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
if x_test.shape[0] == 0 or y_test.shape[0] == 0:
    print("Warning: x_test or y_test is empty. Skipping validation data.")
    history = model.fit(x_train, y_train, epochs=20, batch_size=32)
else:
    history = model.fit(x_train, y_train, epochs=20, batch_size=32, validation_data=(x_test, y_test))

# Predict on test data
predictions = model.predict(x_test)
predictions = scaler.inverse_transform(predictions.reshape(-1,1))

# Get true y_test values from the original data (denormalized)
# Remember, test_data was scaled from original test set
actual_prices = test_data[seq_length:] # Skip first 'seq_length' points
actual_prices = test_data[seq_length:]
if actual_prices.shape[0] == 0:
    print("Warning: Not enough test data. Skipping inverse_transform and plotting.")
else:
    actual_prices = scaler.inverse_transform(actual_prices.reshape(-1,1))

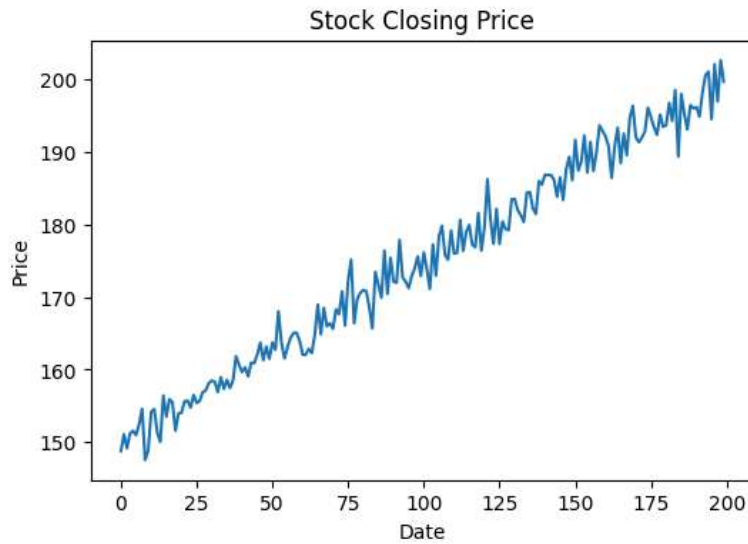
if actual_prices.shape[0] > 0:
    actual_prices = scaler.inverse_transform(actual_prices.reshape(-1,1))
    plt.plot(...)
else:
    print("Skipping plot: No actual prices available.")

print(f"Length of predictions: {len(predictions)}")
print(f"Length of actual_prices: {len(actual_prices)}")
print(f"Index range: {len(df.index[-len(predictions):])}")

min_len = min(len(predictions), len(actual_prices))

if min_len > 0:
    plt.plot(df.index[-min_len:], actual_prices[:min_len], color='blue', label='Actual Price')
    plt.plot(df.index[-min_len:], predictions[:min_len], color='red', label='Predicted Price')
    plt.title('Stock Price Prediction')
    plt.xlabel('Date')
    plt.ylabel('Price')
    plt.legend()
    plt.show()
else:
    print("No data to plot.")

```



```
x_test.shape before reshape: (0,)
Error: x_test is empty or not enough data points for sequence length.
Length of test_data: 40
Not enough data in test set. Please use more data or reduce sequence length.
Error: x_test is empty. Cannot reshape.
Warning: x_test or y_test is empty. Skipping validation data.
Epoch 1/20
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_di
super().__init__(**kwargs)
4/4 ██████████ 5s 93ms/step - loss: 0.3124
Epoch 2/20
4/4 ██████████ 0s 59ms/step - loss: 0.1010
Epoch 3/20
4/4 ██████████ 0s 55ms/step - loss: 0.0088
Epoch 4/20
4/4 ██████████ 0s 53ms/step - loss: 0.0290
Epoch 5/20
4/4 ██████████ 0s 56ms/step - loss: 0.0051
Epoch 6/20
4/4 ██████████ 0s 58ms/step - loss: 0.0125
Epoch 7/20
4/4 ██████████ 0s 54ms/step - loss: 0.0102
Epoch 8/20
4/4 ██████████ 0s 55ms/step - loss: 0.0039
Epoch 9/20
4/4 ██████████ 0s 57ms/step - loss: 0.0045
Epoch 10/20
4/4 ██████████ 0s 63ms/step - loss: 0.0042
Epoch 11/20
4/4 ██████████ 0s 57ms/step - loss: 0.0031
Epoch 12/20
4/4 ██████████ 0s 55ms/step - loss: 0.0032
Epoch 13/20
4/4 ██████████ 0s 54ms/step - loss: 0.0030
Epoch 14/20
4/4 ██████████ 0s 60ms/step - loss: 0.0029
Epoch 15/20
4/4 ██████████ 0s 59ms/step - loss: 0.0031
Epoch 16/20
4/4 ██████████ 0s 61ms/step - loss: 0.0030
Epoch 17/20
4/4 ██████████ 0s 54ms/step - loss: 0.0028
Epoch 18/20
4/4 ██████████ 0s 58ms/step - loss: 0.0027
Epoch 19/20
4/4 ██████████ 0s 56ms/step - loss: 0.0024
Epoch 20/20
4/4 ██████████ 0s 68ms/step - loss: 0.0024
1/1 ██████████ 0s 444ms/step
Warning: Not enough test data. Skipping inverse_transform and plotting.
Skipping plot: No actual prices available.
Length of predictions: 1
Length of actual_prices: 0
Index range: 1
No data to plot.
```

```
min_len = min(len(predictions), len(actual_prices))

# Plot
plt.figure(figsize=(7, 4))
plt.plot(actual_prices[-min_len:], color='blue', label='Actual Price')
plt.plot(predictions[-min_len:], color='red', label='Predicted Price')
plt.title('Stock Price Prediction')
plt.xlabel('Time Step')
plt.ylabel('Price')
plt.legend()
plt.show()
```

